



移动智能终端生态联盟——无障碍标准

无障碍开发适配指导

编制单位：移动智能终端生态联盟·标准工作组

2025 年 3 月

目录

1、前言	4
1.1、文档目的	4
1.2、相关术语	4
1.3、适用对象	4
1.4、适配原则	4
1.4.1、整体原则	4
1.4.2、多语言	5
2、整体介绍	5
2.1、能力构成	5
2.2、操作模式	5
2.3、语音朗读	7
3、TalkBack 适配	9
3.1、基本适配	9
3.1.1、标签内容	9
3.1.2、系统控件	13
3.1.3、焦点控制	15
3.1.4、朗读控制	16
3.2、自定义适配	16
3.2.1、开启状态判断	16
3.2.2、自定义视图适配	18
3.2.3、虚拟视图适配	23
4、无障碍服务适配（AccessibilityService）	29
4.1、Manifests 声明和权限	30
4.1.1、无障碍服务声明	30
4.1.2、无障碍服务配置	30
4.1.3、注册无障碍事件	31
4.2、无障碍服务	31
4.2.1、无障碍服务方法	31
4.2.2、获得事件详情	32
4.2.3、为用户采取措施	33
4.3、注意事项	35
4.3.1、Android12 无障碍服务 isAccessibilityTool 管控	35
5、常见问题	35
5.1、如何判断当前无障碍 talkback 是否开启？	35
5.2、播报如何加标题？ 如：“xxx, 标题”	35
5.3、如何去掉播报 “点按两次即可激活”？	36
5.4、如何直接播报文本？	36
5.5、如何修改 “点按两次即可 xx” 的 ACTION 描述？	36
5.6、如何转换无障碍控件类型或修改控件类型描述	37
5.7、如何去掉冗余的焦点？	37
5.8、如何清除已选中的无障碍焦点？	37
5.9、如何将焦点转移到父布局？	37

5. 10. 如何主动获取焦点?.....	37
5. 11. 如何控制 talkback 焦点顺序.....	37
5. 12. web 相关 H5 页面如何适配?	38
5. 13. 如何打印节点树?	38
6、参考文档.....	38

1、前言

信息无障碍是指任何人（无论是健全人还是残疾人，无论是年轻人还是老年人）在任何情况下都能平等的、方便地、无障碍地获取信息、利用信息。互联网产品通过进行易用性、可用性优化，可以让所有用户获得更好的用户体验，甚至视障者、老年人、读写障碍者、听障者等特殊用户都能顺畅使用。我们 80% 的信息获取来源于视觉，所以优先从视觉切入信息无障碍，实现产品和读屏软件兼容。

读屏软件也叫屏幕阅读器，Android 有 Talkback，ios 有 voiceover（旁白），是协助用户使用应用功能的工具，将屏幕信息转化为语音，引导交互。所以，为了实现视觉无障碍需要各功能模块去适配 Talkback 工具。

通过产品的无障碍服务，视障人士、听障人士、老年人等障碍群体能够正常获得并使用信息，给用户提供更多的人文关怀，从而提升品牌美誉度。

1.1、文档目的

编制无障碍的相关适配规范和介绍，为应用模块适配无障碍服务提供一些指导与参考。

1.2、相关术语

- **信息无障碍**

信息无障碍是指任何人（无论是健全人还是残疾人，无论是年轻人还是老年人）在任何情况下都能平等的、方便地、无障碍地获取信息、利用信息。

- **Talkback**

Talkback 是一款由 Google 官方开发的系统工具软件，它的定位是帮助盲人或者有视力障碍的用户提供语言辅助。让盲人或者视力受损的用户可以非常方便的与他们的设备进行有效的互动。当使用 Android 自带的程序时，Talkback 会实时的提供语言反馈。

1.3、适用对象

移动开发过程中应用开发迭代相关的需求策划工程师，软件开发人员，测试人员等应用开发团队。

1.4、适配原则

1.4.1、整体原则

可感知性：信息和用户界面组件必须以可感知的方式呈现给用户。【talkback 模式下控件可朗读】

可操作性：用户界面组件和导航必须可操作。【talkback 模式下控件易操作】

可理解性：信息和用户界面操作必须是可理解的。【talkback 模式下朗读内容&操作引导】

鲁棒性：内容必须健壮到可信地被种类繁多的用户代理（包括辅助技术）所解释。【适配 talkback】

1.4.2、多语言

- 国内至少适配中英文
- 海外所有多语言均需要适配

2、整体介绍

2.1、能力构成

a) Android Accessibility Suite ([Android 无障碍套件](#))：TalkBack 在 Android 中，被 Google 打包为 [Android 无障碍套件](#) 内置系统对外提供服务，其中包括无障碍菜单、随选朗读、开关控制和 TalkBack。我们可以在搭载 Android 9 及更高版本的设备上找到 Android 无障碍套件并且可以在 GooglePlay 商店在线更新。

b) TTS 语音包：将读屏信息进行**语音播报**，目前国内主要有：讯飞语音引擎、vivo AIService TTS 引擎等。海外主要内置 Google tts 引擎。

类型	TTS 语音包
海外	Google Text-to-speech Engine
国内	厂商 TTS（如 Vivo AIService TTS）
	讯飞语音引擎

2.2、操作模式

在 TalkBack 模式下，用户操作模式发生变化，只能通过使用**滑动浏览**（左右滑动）或**触摸浏览**（单指点按）控制**焦点**（绿框）位置，从而通过语音反馈描述操作的结果（如启动一个 app）和事件（如通知）。

目前手势交互模型为：

- 1、一根手指**缓慢移动**执行触摸探索。

- 2、一根手指**快速移动**来执行手势。
- 3、**两个靠近的手指**向**同一方向**移动执行拖动操作。
- 4、**多指手势**传递到视图层次结构。
- 5、向**不同方向**移动的**两个手指**被视为多指手势。
- 6、**双击**可在可访问性焦点矩形上执行单击操作。
- 7、**点按并按住**一段时间执行长按，方式与上面的点击类似。

不同的手势默认对应不同的操作（注意：talkback 设置页面可自定义手势对应的操作：talkback-设置-自定义手势），详情查看[使用 TalkBack 手势](#)。

+

手势与操作对应关系举例

操作	单指手势	多指手势
移至屏幕上的下一项内容	向右滑动	--
移至屏幕上的上一项内容	向左滑动	--
触摸浏览：读出手指轻触的内容	在屏幕上拖动手指	--
选择一项内容	点按	--
移至下一个朗读控件	先向上再向下滑动	用三指向下滑动
移至上一个朗读控件	先向下再向上滑动	用三指向上滑动
更改朗读控件中所选内容的值	向上或向下滑动	--
跳转到下一项内容 ○ 遵循朗读控件设置	向下滑动	--
跳转到上一项内容 ○ 遵循朗读控件设置	向上滑动	--
从下一项开始朗读	--	用双指点按三次
启用“屏幕搜索”功能	先向左再向下滑动	--
放大功能设置	点按三次	--

操作	手势	多指手势
向上或向下滚动	用双指向上或向下滑动	--
向左或向右滚动	用双指向左或向右滑动	--

操作	手势
启用	点按两次
长按聚焦的内容	点按两次并按住
向上移动滑块 ○ 例如：音量	向上滑动，或者先向右再向左滑动
向下移动滑块 ○ 例如：音量	向下滑动，或者先向左再向右滑动

部分操作触发的时候应用侧可以进行适配调整产品表现，操作列表查看
<https://developer.android.com/reference/androidx/core/view/accessibility/AccessibilityNodeInfoCompat.AccessibilityActionCompat>

2.3、语音朗读

1、借助**朗读控件**（tts 语音引擎），我们可以更改 TalkBack 的朗读速度，朗读音高，以及朗读的语言。

2、在 TalkBack 设置内，我们可以对朗读音频详细程度做自定义控制，包含**使用提示、元素类型、元素 ID、键盘提示**以及**是否读出密码**等都可以控制是否朗读

+

TalkBack 设置音频详细程度



3、在 TalkBack 高级设置内我们还可以对元素说明朗读顺序进行调整，状态、名称、类型→类型、名称、状态等。

+

元素说明朗读顺序



3、TalkBack 适配

3.1、基本适配

3.1.1、标签内容

3.1.1.1、内容描述

应用中的每个互动界面元素都应包含描述该元素用途的说明。对于基本控件，我们可以在元素的 `contentDescription` 属性中添加此说明。TextView 或者继承至其的控件, 如果 `contentDescription` 属性的值为空，无障碍服务会获取 `text` 属性的文本信息作为语音提示。

contentDescription 示例

```
<!-- Use string resources for easier localization. -->
<!-- The en-US value for the following string is "Inspect". -->
<ImageView
    ...
    android:contentDescription="@string/inspect" />

//or java code
view.setContentDescription(R.string.accessibility_custom_on);
```

3.1.1.2、状态描述

应用中的部分互动界面元素可能会有不同状态，一般我们可以通过 `setStateDescription` 接口设置 `stateDescription` 作为控件状态描述。

3.1.1.3、可修改的元素

为可修改的元素（如 `EditText` 对象）添加标签时，提供 `android:hint` 属性代替内容描述，文本区域为空的时候此属性帮助用户理解应该输入什么样的内容。当文本区域填充上内容，TalkBack 将会读出输入的文本，而不会读出提示文本。

hint 示例

```
<!-- The hint text for en-US locale would be
    "Apartment, suite, or building". -->
<EditText
    android:id="@+id/addressLine2"
    android:hint="@string/aptSuiteBuilding" ... />
```

此外，对于给定的 `EditText` 元素通常会有一个相应的 `View` 对象，该对象描述了用户应在 `EditText` 元素中输入的内容，Android 提供 `View` 对象的 `android:labelFor` 属性来指明这种关系。在朗读 `EditText` 元素时会以“**view**”对应的“`EditText`”的形式朗读元素。

labelFor 示例

```
<!-- Label text for en-US locale would be "Username:" -->
<TextView
    android:id="@+id/usernameLabel" ...
    android:text="@string/username"
    android:labelFor="@+id/usernameEntry" />

<EditText
    android:id="@+id/usernameEntry" ... />

<!-- Label text for en-US locale would be "Password:" -->
<TextView
    android:id="@+id/passwordLabel" ...
```

```

    android:text="@string/password
    android:labelFor="@+id/passwordEntry" />

```

```

<EditText
    android:id="@+id/passwordEntry"
    android:inputType="textPassword" ... />

```

3.1.1.4、集合中的元素

向集合中的元素添加标签时，每个标签都应该是唯一的。这样，系统的无障碍服务在读出标签时就可以特指屏幕上的 1 个元素。这种对应关系可让用户知道他们何时循环浏览界面，或何时将焦点移到了已经发现的元素。

Adapter 适配

```

public class MovieRating {
    private String title;
    private int starRating;
    // ...
    public String getTitle() { return title; }
    public int getStarRating() { return starRating; }
}

public class MyMovieRatingsAdapter
    extends RecyclerView.Adapter<MyAdapter.MyRatingViewHolder> {
    private MovieRating[] myData;

    public static class MyRatingViewHolder extends RecyclerView.ViewHolder {
        public ImageView ratingView;
        public MyRatingViewHolder(ImageView iv) {
            super(iv);
            ratingView = iv;
        }
    }

    @Override
    public void onBindViewHolder(MyRatingViewHolder holder, int position) {
        MovieRating ratingData = myData[position];
        holder.ratingView.setContentDescription("Movie " + position + ": " +
            ratingData.getTitle() + ", " + ratingData.getStarRating() +
            " stars")
    }
}

```

3.1.1.5、视图容器 (ViewGroup)

如果应用显示的多个界面元素构成一个自然组（如歌曲的详细信息或消息的属性），应将这些元素整理到一个容器中，该容器通常是 `ViewGroup` 的子类。将容器对象的 `android:screenReaderFocusable` 或者 `android:focusable` 属性设为 `true`，并将每个内部对象的 `android:focusable` 属性设为 `false`。这样，无障碍服务就可以在单次语音中逐个读出内部元素的内容说明。这样整合相关元素有助于使用辅助技术的用户更高效地发现屏幕上的信息。

ViewGroup 示例

```
<!-- In response to a single user interaction, accessibility services announce
      both the title and the artist of the song. -->
<ConstraintLayout
    android:id="@+id/song_data_container" ...
    android:screenReaderFocusable="true"
    android:focusable="true">

    <TextView
        android:id="@+id/song_title" ...
        android:focusable="false"
        android:text="@string/my_song_title" />
    <TextView
        android:id="@+id/song_artist"
        android:focusable="false"
        android:text="@string/my_songwriter" />
</ConstraintLayout>
```

以上操作会将来自其子 `View` 的内容标签合并在一起，如果需要抑制此行为，并指明您希望为该项及其不可聚焦的子 `View` 提供自己的说明，请在 `ViewGroup` 上设置 `contentDescription`。比如有一个展示型卡片，不做任何设置时，可能实际无障碍自动播报的顺序或播报的内容和预期不符合，可以 `format` 需要播报的内容，给最外层 `view` 整体设置 `contentDescription`。

ViewGroup contentDescription 示例

```
<!-- In response to a single user interaction, accessibility services
      announce the custom content description for the group. -->
<ConstraintLayout
    android:id="@+id/song_data_container" ...
    android:screenReaderFocusable="true"
    android:contentDescription="@string/title_artist_best_song">

    <TextView
        android:id="@+id/song_title" ...

        <!-- Content ignored by accessibility services -->
        android:text="@string/my_song_title" />
    <TextView
        android:id="@+id/song_artist"
```

```
<!-- Content ignored by accessibility services -->
    android:text="@string/my_songwriter" />
</ConstraintLayout>
```

3.1.1.6、标题

如果特定的 View 元素表示一个标题，您可以通过将该元素的 [android:accessibilityHeading](#) 属性设为 true，表明它的无障碍服务用途。在 TalkBack 朗读时会在最后附加描述“**标题**”，无障碍服务的用户可以选择浏览标题，而不是浏览段落或字词。这种灵活性可改善文字浏览体验。

“标题”样例代码

```
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.P) {
    mBbkTitleView.getCenterView().setAccessibilityHeading(true);
}
```

注意：要加版本判断，否则自动化测试低于 Android9 会崩溃

3.1.2、系统控件

在设计应用的界面时，应尽量使用或扩展位于 Android 类层次结构中尽可能靠下位置的系统组件。位于层次结构最下方的系统控件已经具备应用所需的大多数无障碍功能。扩展系统提供的这些控件比根据更通用的 [View](#)、[ViewCompat](#)、[Canvas](#) 和 [CanvasCompat](#) 类创建您自己的控件更容易。如果必须直接扩展 View 或 Canvas（这可能是高度个性化体验或游戏关卡所必需的），请参阅 3.2 章节 [3.2、自定义适配或让自定义视图使用起来更没有障碍](#)。

接下来以 [Switch](#) 为例讲解一种特殊的 Switch [TriSwitch](#) 的适配过程。[TriSwitch](#) 对象的工作方式与 [Switch](#) 对象类似，不同之处在于，[TriSwitch](#) 的每个实例允许用户在 3 种可能的状态之间切换。

3.1.2.1、继承关系

```
View
└─ TextView
   └─ Button
      └─ CompoundButton
         └─ Switch
```

新的 [TriSwitch](#) 类最好直接从 [Switch](#) 类扩展。这样，[Android 无障碍功能框架](#) 就可以提供 [TriSwitch](#) 类所需的大多数无障碍功能：

- **无障碍操作**（[AccessibilityNodeInfo.AccessibilityAction](#)）：告知系统无障碍服务如何模拟在 [TriSwitch](#) 对象上执行的各种可能的用户输入。（继承自 [View](#)。）

- **无障碍事件 ([AccessibilityEvent](#))**：告知无障碍服务当屏幕刷新或更新时，TriSwitch 对象的外观会发生的各种可能的变化方式。（继承自 View。）
- **特征内容**：有关每个 TriSwitch 对象的详细信息，例如其显示的任何文字的内容。（继承自 TextView。）
- **状态信息**：TriSwitch 对象的当前状态的说明，如“选中”或“未选中”。（继承自 CompoundButton。）
- **状态的文字说明**：文字类说明，解释了各种状态的含义。（继承自 Switch。）

继承自 Switch 及其父类的这种聚合行为几乎就是 TriSwitch 对象的预期行为。因此，我们的实现可以把重点放在将状态数从 2 种扩展到 3 种上。

3.1.2.2、自定义事件处理

当我们扩展某个系统控件时，可能会改变用户与该控件交互方式的某一方面。请务必定义这些互动变化，以便无障碍服务可以更新应用的微件，就像用户直接与微件互动一样。

一般准则是，对于我们替换的每个基于视图的回调，还需要通过替换 [ViewCompat.replaceAccessibilityAction\(\)](#) 来重新定义相应的无障碍操作。在应用的测试中，您可以通过调用 [ViewCompat.performAccessibilityAction\(\)](#) 来验证这些重新定义的操作的行为。

在当前的例子中，与普通的 Switch 对象不同，点按 TriSwitch 对象可以循环切换 3 种可能的状态。因此，需要更新相应的 ACTION_CLICK 无障碍操作。

replaceAccessibilityAction

```
public class TriSwitch extends Switch {
    // 0, 1, or 2.
    private int currentState;

    public int getCurrentState() {
        return currentState;
    }

    public TriSwitch() {
        updateAccessibilityActions();
    }

    private void updateAccessibilityActions() {
        ViewCompat.replaceAccessibilityAction(this, ACTION_CLICK,
            action-label, (view, args) -> moveToNextState());
    }

    private void moveToNextState() {
        currentState = (currentState + 1) % 3;
    }
}
```

```
}
```

3.1.3、焦点控制

3.1.3.1、启用或屏蔽焦点

如果界面中某个元素的存在只是为了让内容看起来间距合理或布局美观，请将其 `android:contentDescription` 属性设为 "null"。

如果应用仅支持搭载 Android 4.1（API 级别 16）或更高版本的设备，我们可以将这些纯装饰性元素的 `android:importantForAccessibility` 属性设为 "no"，该属性默认为 "yes"。

- **focusable**
当用户界面元素的 `android:focusable` 属性设置为 true 时，可以使用定向控制到达该元素。该设置允许用户使用**定向控制**聚焦元素并与之交互。Android 框架提供的用户界面控件默认可聚焦，并通过改变控件的外观可直观表明其聚焦。
- **focusableInTouchMode**
`android:focusableInTouchMode` 属性是代表在**触摸模式**下是否可聚焦，通常这个值也应该设置为 true，因为视障用户通过 TalkBack 使用应用时，多数是使用触摸屏设备的。
- **importantForAccessibility**
`android:importantForAccessibility` 属性有 3 个状态一个是 no、yes、auto。No 是代表对此控件对无障碍服务不重要（不朗读），yes 代表此控件对无障碍服务此控件重要，auto 是代表此控件对于无障碍服务的重要性由系统判断。

3.1.3.2、主动获取焦点

希望一个 view 获取 talkback 的焦点，可以在布局加载完成后在 UI 线程使用方法 `view.sendAccessibilityEvent(AccessibilityEvent.TYPE_VIEW_HOVER_ENTER)`；触发聚焦。

TYPE_VIEW_HOVER_ENTER 样例代码

```
mView.postDelayed(new Runnable() {  
    @Override  
    public void run() {  
        mView.sendAccessibilityEvent(AccessibilityEvent.TYPE_VIEW_HOVER_ENTER);  
    }  
}, 100);
```

3.1.3.3、焦点导航

定向控制器导航

当用户在任何方向使用定向控制器导航时，焦点从一个用户界面元素（视图）传递到另一个由焦点顺序指定的用户界面元素（视图）。焦点顺序是以一种在某一特定方向上寻找相邻元素的算法为基础的。

在极少数情况下，默认算法可能不匹配开发者定义的顺序，或可能对于用户不符合逻辑。在这些情况下，可以在布局文件中使用下列的 xml 属性明确地覆盖焦点顺序：

- `android:nextFocusDown`
- 当用户向下导航时，定义下一个接收焦点的视图；
- `android:nextFocusLeft`
- 当用户向左导航时，定义下一个接收焦点的视图；
- `android:nextFocusRight`
- 当用户向右导航时，定义下一个接收焦点的视图；
- `android:nextFocusUp`
- 当用户向上导航时，定义下一个接收焦点的视图。

无障碍服务导航

无障碍焦点顺序需要用 `android:accessibilityTraversalAfter` 和 `android:AccessibilityTraversalBefore` 这两个属性来控制。

这两个属性的值都是控件 `id`。其中 `accessibilityTraversalBefore` 属性代表本控件的顺序在 `id` 指定的控件之前，`accessibilityTraversalAfter` 属性代表本控件的焦点顺序在 `id` 指定的控件的焦点之后。

3.1.4、朗读控制

3.1.4.1、内容改变

如果用户在正常使用应用的过程中，应用中的按钮或其他控件的功能会发生变化（例如，一个按钮从**播放**变为**暂停**），保证按钮的 [android:contentDescription](#) 会相应改变。

3.1.4.2、补充音频反馈

仅使用 Android 无障碍框架为应用程序提供无障碍音频反馈。使用 [android:contentDescription](#) XML 布局属性或使用无障碍框架 APIs 动态地添加该属性来为应用提供提示信息。

例如，如果开发者想要把应用程序执行的操作告知用户，如书籍自动翻页功能，使用 [announceForAccessibility\(CharSequence\)](#) 方法让无障碍服务为用户读出该信息。

3.1.4.3、视图组合朗读

如果控件比推荐的触摸尺寸小，可以考虑使用 [ViewGroup](#) 将这些控件组合起来，并使用 [android:contentDescription](#) 为该组合提供内容描述。

3.2、自定义适配

3.2.1、开启状态判断

- 无障碍辅助功能整体状态

通过 AccessibilityManager 判断 isEnabled 状态, 为 True 表示系统内有辅助功能服务开启, 为 False 表示系统内没有辅助功能服务开启。**注意: 无障碍辅助功能开启并不代表 talkback 开启, 系统有其他应用也可以开启无障碍辅助功能**

AccessibilityManager.isEnabled 代码示例

```
AccessibilityManager am =
    (AccessibilityManager) getSystemService (ACCESSIBILITY_SERVICE);
boolean isAccessibilityEnabled = am.isEnabled();
```

- 无障碍触摸浏览状态

通过 AccessibilityManager 判断 isTouchExplorationEnabled 状态, 为 True 表示系统内有拥有触摸浏览能力的辅助功能服务开启, 为 False 表示系统内没有拥有触摸浏览特性的辅助功能服务开启。TalkBack 服务是拥有触摸浏览特性的辅助功能服务。

AccessibilityManager.isTouchExplorationEnabled 代码示例

```
AccessibilityManager am =
    (AccessibilityManager) getSystemService (ACCESSIBILITY_SERVICE);
boolean isTouchExplorationEnabled = am.isTouchExplorationEnabled();
```

- talkback 无障碍服务状态

【重要】通过获取当前系统开启的所有辅助功能服务的 Settings 值, 判断其中有没有包含 talkback, 从而判断 talkback 开关有没有开启

```
/*
 * 获取当前开启的所有辅助功能服务
 */
public static Set<ComponentName> getEnabledServicesFromSettings(Context context, int
userId) {
    String enabledServicesSetting =
Secure.getStringForUser(context.getContentResolver(),
"enabled_accessibility_services", userId);
    if (enabledServicesSetting == null) {
        return Collections.emptySet();
    } else {
        Set<ComponentName> enabledServices = new HashSet();
        SimpleStringSplitter colonSplitter = sStringColonSplitter;
        colonSplitter.setString(enabledServicesSetting);

        while(colonSplitter.hasNext()) {
            String componentNameString = colonSplitter.next();
            ComponentName enabledService =
ComponentName.unflattenFromString(componentNameString);
            if (enabledService != null) {
```

```

        enabledServices.add(enabledService);
    }
}

return enabledServices;
}
}

```

本质是获取 `enabled_accessibility_services` 数据库字段的值,用 `:` 作为分隔符分割字符串,分割出来的就是对应的辅助功能服务。

如果包含

`com.google.android.marvin.talkback/com.google.android.marvin.talkback.TalkBackService` (或者 `com.google.android.marvin.talkback/.TalkBackService`) 则表示 talkback 开关打开的

3.2.2、自定义视图适配

如果应用需要一个[自定义视图组件](#),开发者必须做一些额外的工作来确保自定义视图是无障碍可访问的。这些都是确保视图可访问性的主要任务:

- 处理定向控制器的点击;
- 实现无障碍 API 方法;
- 发送特定的 [AccessibilityEvent](#) 对象到自定义视图;
- 为视图填充 [AccessibilityEvent](#) 和 [AccessibilityNodeInfo](#);

3.2.2.1、处理定向控制器的点击

在大多数的 Android 设备上,使用定向控制器单击视图会发送一个带有 [KEYCODE_DPAD_CENTER](#) 的 [KeyEvent](#) 事件到当前的焦点视图。所有的标准 Android 视图已经适当地处理 [KEYCODE_DPAD_CENTER](#)。当构建一个自定义 [View](#) 控件,确保这个事件的效果跟在触摸屏上触摸视图的效果一样。

自定义控件也应该将 [KEYCODE_ENTER](#) 事件作为 [KEYCODE_DPAD_CENTER](#) 事件处理,这种方法使全键盘用户的交互更加容易。

3.2.2.2、实现无障碍 API 方法

无障碍事件是用户与应用中与视觉界面元素的交互消息。这些消息是由无障碍服务 ([Accessibility Services](#)) 处理的,这些服务使用这些事件中的信息去产生[补充反馈和提示](#)。扩展无障碍 API 方法是 [View](#) 类和 [View.AccessibilityDelegate](#) 类的一部分。

如果应用程序目标版本是 **Android 4.0 (API 级别 14) 或更高**,就直接在自定义视图类中重写并实现上面列出的无障碍方法 (也可以使用 `ViewCompat.setAccessibilityDelegate` 来设置对应方法,效果一样)。这些方法如下:

- [sendAccessibilityEvent\(\)](#) (API 级别 4)

当用户操作视图时，这个方法被调用。该事件使用用户操作类型进行分类，例如 [TYPE_VIEW_CLICKED](#)。开发者一般不需要实现这个方法，除非创建了自定义视图。

- [onRequestSendAccessibilityEvent\(\)](#) (API 级别 14)

当视图中的一个子视图生成 [AccessibilityEvent](#) 时，系统调用这个方法。该步骤允许父视图使用额外信息修复无障碍事件。当自定义视图有子视图，且其父视图可以为无障碍事件提供对无障碍服务有用的上下文信息的时候，此时开发者应该实现这个方法。

- [onPopulateAccessibilityEvent\(\)](#) (API 级别 14)

该方法为视图的 [AccessibilityEvent](#) 设置 [朗读文本提示](#)。如果视图是另一个生成无障碍事件视图的子视图，调用该方法。

注意： 通过该方法修改文本以外的属性可能会被其他方法重写。虽然在该方法中开发者可以修改无障碍事件的属性，但是应该做到这些改变仅限于文本内容，并使用 [onInitializeAccessibilityEvent\(\)](#) 方法修改事件的其他属性。

注意： 如果该事件的实现完全重写了输出文本，而不允许布局的其他部分修改这个内容，不要在代码中调用该方法的父类实现。

- [onInitializeAccessibilityEvent\(\)](#) (API 级别 14)

系统调用此方法来获取 [超出文本内容的视图状态的附加信息](#)。如果自定义视图提供简单 [TextView](#) 或 [Button](#) 以外的交互控制，开发者应该重写该方法，并且使用该方法设置该视图的额外信息到事件中，如密码区域类型、复选框类型或提供用户交互或反馈的状态。如果重写这个方法，开发者必须调用它父类的实现方法，然后只修改那些父类中尚未设置的属性。

- [onInitializeAccessibilityNodeInfo\(\)](#) (API 级别 14)

该方法为无障碍服务提供 [视图的状态信息](#)。默认 [View](#) 的实现包含一组标准的视图属性，但是如果自定义视图提供了超出 [TextView](#) 或 [Button](#) 的交互，开发者应该重写该方法，并在 [AccessibilityNodeInfo](#) 对象中设置视图的额外信息，该对象被该方法处理。

3.2.2.3、发送无障碍事件

根据自定义视图的特性，不是由默认实现处理的事件，可能需要在不同时刻发送 [AccessibilityEvent](#) 对象。[View](#) 类为这些事件类型提供了默认的实现方法：

- 自 API 级别 4 引入：
 - [TYPE_VIEW_CLICKED](#)
 - [TYPE_VIEW_LONG_CLICKED](#)
 - [TYPE_VIEW_FOCUSED](#)
- 自 API 级别 14 引入：
 - [TYPE_VIEW_SCROLLED](#)

- [TYPE_VIEW_HOVER_ENTER](#)
- [TYPE_VIEW_HOVER_EXIT](#)

注意：悬停（Hover）事件与触摸浏览相关联，触摸浏览使用这些事件作为触发器，为用户界面元素提供音频反馈。

一般来说，每当自定义视图的内容发生变化时，应该发送一个 [AccessibilityEvent](#) 事件。例如，如果创建了一个自定义的滑动条，用户可以按下左边或右边的箭头选择数值，当滑动条的值发生改变时，自定义视图应该发出一个 [TYPE_VIEW_TEXT_CHANGED](#) 类型的事件。

sendAccessibilityEvent() 代码示例

```
@Override
public boolean onKeyUp (int keyCode, KeyEvent event) {
    if (keyCode == KeyEvent.KEYCODE_DPAD_LEFT) {
        mCurrentValue--;
        sendAccessibilityEvent(AccessibilityEvent.TYPE_VIEW_TEXT_CHANGED);
        return true;
    }
    ...
}
```

3.2.2.4、填充无障碍事件

每个 [AccessibilityEvent](#) 有一组必需的属性，这些属性描述当前视图的状态。这些属性包括视图类名称、内容描述和检查状态等信息。每个事件类型必须的特定属性被描述在 [AccessibilityEvent](#) 参考文档中。[View](#) 实现提供这些属性的默认值。包含类名和事件时间标记在内的很多值会被自动提供。如果正在创建一个自定义视图组件，必须提供一些关于视图内容和特性的信息。这些信息可能是简单的按钮标签，但是也可能包含想要添加到事件中的额外的状态信息。

在 Android 4.0 (API 级别 14) 和更高的系统中，使用 [onPopulateAccessibilityEvent\(\)](#) 和 [onInitializeAccessibilityEvent\(\)](#) 方法来填充或修改 [AccessibilityEvent](#) 事件中的信息。[onPopulateAccessibilityEvent\(\)](#) 方法可专门用来为事件添加或修改文本内容，这些信息会被如 TalkBack 的无障碍服务转化为音频反馈。使用 [onInitializeAccessibilityEvent\(\)](#) 方法添加事件的其他信息，比如视图的选择状态。

此外，还应该实现 [onInitializeAccessibilityNodeInfo\(\)](#) 方法。该方法填充 [AccessibilityNodeInfo](#) 对象，视图层次在接收此事件后生成无障碍事件，无障碍服务使用 [AccessibilityNodeInfo](#) 对象访问该视图层次，获得更多的上下文信息并为用户提供合适的反馈。比如填充控件类型可以使用 [AccessibilityNodeInfoCompat.setRoleDescription\(\)](#) 方法，或者 [AccessibilityNodeInfo.getExtras\(\).putCharSequence\("AccessibilityNodeInfo.roleDescription", "我是控件类型"\)](#)；

无障碍事件填充示例代码

```
public static class AccessibleCompoundButtonInheritance extends BaseToggleButton {
```

```
public AccessibleCompoundButtonInheritance(Context context, AttributeSet
attrs) {
    super(context, attrs);
}

boolean mDownTouch = false;

@Override
public void onInitializeAccessibilityEvent(AccessibilityEvent event) {
    super.onInitializeAccessibilityEvent(event);
    //调用父类实现让父类设置事件属性。然后添加父类不支持的新属性。
    event.setChecked(isChecked());
}

@Override
public void onPopulateAccessibilityEvent(AccessibilityEvent event) {
    super.onPopulateAccessibilityEvent(event);
    //调用父类实现来填充事件的文本。然后添加父类中不存在的文本。
    //常常只需要在自定义视图中添加这些文本。
    CharSequence text = getText();
    if (!TextUtils.isEmpty(text)) {
        event.getText().add(text);
    }
}

@Override
public void onInitializeAccessibilityNodeInfo(AccessibilityNodeInfo info) {
    super.onInitializeAccessibilityNodeInfo(info);
    // We called the super implementation to let super classes set
    // appropriate info properties. Then we add our properties
    // (checkable and checked) which are not supported by a super class.
    //info.setSelected(isChecked());
    info.setCheckable(true);
    info.setChecked(isChecked());
    //info.setClickable(true);
    // Very often you will need to add only the text on the custom view.
    CharSequence text = getText();
    if (!TextUtils.isEmpty(text)) {
        info.setText(text);
        info.setContentDescription(text);
    }
}
```

```
info.getExtras().putCharSequence("AccessibilityNodeInfo.roleDescription", "我是控件类型");

//info.setStateDescription("我是状态描述");
//info.setHintText("提示文本");
//info.setContentDescription("我是内容描述");
// 给滚轮选取器添加读屏特定手势

info.addAction(AccessibilityNodeInfo.AccessibilityAction.ACTION_SCROLL_BACKWARD);

info.addAction(AccessibilityNodeInfo.AccessibilityAction.ACTION_SCROLL_FORWARD);
}

@Override
public boolean performAccessibilityAction(int action, Bundle arguments) {
    // 拦截在上一步中添加的读屏手势
    // 并处理对应手势下的滚轮选取逻辑
    switch (action) {
        case AccessibilityNodeInfo.ACTION_SCROLL_BACKWARD:
            setTotalScrollY(--mA11ySelectedItem * itemHeight);
            break;
        case AccessibilityNodeInfo.ACTION_SCROLL_FORWARD:
            setTotalScrollY(++mA11ySelectedItem * itemHeight);
            break;
        default:
            return super.performAccessibilityAction(action, arguments);
    }
    // 通知重绘视图
    invalidate();
    // 当滚轮选取器值发生变化时
    // 普通用户可以通过动画感知到变化过程
    // 同样，视障者也需要感知这个变化过程
    // 可调用 View.announceForAccessibility(...) 方法
    // 让读屏软件实时将变化的值朗读给用户
    this.postDelayed(new Runnable() {
        @Override
        public void run() {

            announceForAccessibility(adapter.getItem(selectedItem).toString() + label);
        }
    }, 2); // 延迟 2ms 保证界面已经重绘完成
    return true;
}
}
```

我们还可以为 [AccessibilityNodeInfo](#) 对象添加某些读屏特定手势或者移除特定手势,同时覆写 `performAccessibilityAction` 方法拦截对应无障碍手势执行对应操作。

AccessibilityNodeInfo.removeAction 移除对应无障碍操作

```
@Override
public void onInitializeAccessibilityNodeInfo(AccessibilityNodeInfo info) {
    super.onInitializeAccessibilityNodeInfo(host, info);

    info.removeAction(AccessibilityNodeInfoCompat.AccessibilityActionCompat.ACTION_CLICK);

    info.setClickable(false);
}
```

状态表示

- **selected**
有时候会用控件不同的视觉表现（如颜色、形状）来表示控件的选中状态，这时候非视觉用户就无法知道控件的选中状态，开发者需要在无障碍属性中设置 `selected` 选中状态告知用户。
- **checked**
有时候会用图形按钮或自定义控件做出更美观的复选框，这时候非视觉用户由于无障碍属性中没有设置选中状态（`checked`）而不知道这些控件的当前状态是什么。配合 `setCheckable` 朗读“未选中”“点按两次可切换”
- **若需要完全自定义状态，可使用 `setStateDescription` 接口设置状态描述，该接口与 `setContentDescription` 类似，区别是 `contentDescription` 是内容描述**

3.2.3、虚拟视图适配

如果应用程序包含一个具有高视觉特性的[自定义控件](#)（例如日历控件），默认无障碍服务处理不会为用户提供充足的内容描述,开发者可以考虑使用 [AccessibilityNodeProvider](#) 为控件提供[虚拟视图层次](#)。

在日历的场景中，整个日历是作为独立 View 实现，因此如果不采取其他措施，无障碍服务就不能获得有关[视图和视图中用户选择的足够信息](#)。例如，如果用户点击了包含 17 的日期，无障碍框架只能接收到整个日历控件的描述信息。这种情况下，TalkBack 无障碍服务会简单读出“日历”，或者稍好一些的读出“四月日历”，而用户将会不知道自己选择了哪天。

在像这样的情况下，为了给无障碍服务提供足够的上下文信息，Android 框架提供了一种方法——指定虚拟视图层次。应用开发者可以使用[虚拟视图层次](#)方法为无障碍服务提供一个附加视图层次，该层次能最大限度的接近屏幕上的真实信息。这种方法允许无障碍服务给用户提供更有用的上下文信息。

另一个需要虚拟视图继承结构的情况是，用户界面包含一组功能密切相关的控件（视图），**操作一个控件会影响一个或多个组内其他元素的内容**，例如增加按钮和减少按钮分开的数值选择器。在这种情况下，无障碍服务无法获得足够的信息，因为一个控件的操作会改变另一个控件

的内容,且无障碍服务无法获得这种关系。为了处理这种情况,将包含视图的相关控件进行分组,从该容器中提供虚拟视图继承,清楚呈现控件提供的信息和行为。

为了提供虚拟视图继承,在自定义视图或者视图组中重写 [getAccessibilityNodeProvider\(\)](#) 方法,并返回 [AccessibilityNodeProvider](#) 的实现。该无障碍特性实现的一个例子,详见 ApiDemos 样例工程中的 `AccessibilityNodeProviderActivity`。在 Android 1.6 和以后的版本中,可以使用 [Support Library](#) 中的 [ViewCompat.getAccessibilityNodeProvider\(\)](#) 方法,并提供 [AccessibilityNodeProviderCompat](#) 实现,就可以兼容虚拟视图继承。

[ExploreByTouchHelper](#) 类包装了 [AccessibilityNodeProviderCompat](#) 并简化了将自定义视图的逻辑结构信息暴露给无障碍服务的过程,我们可以使用它来帮助我们完成虚拟视图开发适配。

3.2.3.1、虚拟节点生成

通过在自定义 view 里设置 `ViewCompat.setAccessibilityDelegate(this, accessibilityDelegate);` 扩展各个无障碍方法,实现自定义的无障碍。[ExploreByTouchHelper](#) 继承自 `AccessibilityDelegateCompat` 类,并且对 [AccessibilityNodeProviderCompat](#) 进行了包装。

```
public class ExploreByTouchHelperActivity extends Activity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.explore_by_touch_helper);
        ...
    }

    public static class CustomView extends View {
        ...
        private CustomViewTouchHelper mTouchHelper;

        public CustomView(Context context, AttributeSet attrs) {
            super(context, attrs);

            // Set up accessibility helper class.
            mTouchHelper = new CustomViewTouchHelper(this);
            ViewCompat.setAccessibilityDelegate(this, mTouchHelper);
        }
        @Override
        public boolean dispatchHoverEvent(MotionEvent event) {
            // Always attempt to dispatch hover events to accessibility first.
            if (mTouchHelper.dispatchHoverEvent(event)) {
                return true;
            }
        }
    }
}
```



```

    }
    return super.dispatchHoverEvent(event);
}
...
}

private class CustomViewTouchHelper extends ExploreByTouchHelper {
    private final Rect mTempRect = new Rect();

    public CustomViewTouchHelper(View forView) {
        super(forView);
    }

    @Override
    protected int getVirtualViewAt(float x, float y) {
        // 我们还会在 onTouchEvent() 中执行命中检测，我们可以在这里重用该逻辑。这将确保无障碍是打开还是关闭的一致性。
        final int index = getItemIndexUnder(x, y);
        if (index == NO_ITEM) {
            return ExploreByTouchHelper.INVALID_ID;
        }
        return index;
    }

    @Override
    protected void getVisibleVirtualViews(List<Integer> virtualViewIds) {
        // 由于每个项目都应该是可见的，而且我们直接从项目索引映射到虚拟视图ID，我们可以添加每个没有父视图的索引。
        final int n = mItems.size();
        for (int i = 0; i < n; i++) {
            if (mItems.get(i).mParent == null) {
                virtualViewIds.add(i);
            }
        }
    }

    @Override
    protected void onPopulateEventForVirtualView(
        int virtualViewId, AccessibilityEvent event) {
        final CustomItem item = getItem(virtualViewId);
        if (item == null) {
            throw new IllegalArgumentException("Invalid virtual view id");
        }
    }
}

```

// 该事件必须使用 `getText().add()` 或 `setContentDescription()` 填充文本。由于项目的描述是直观显示的，我们将其添加到事件文本中。如果它仅用于无障碍，我们将使用 `setContentDescription()`。

```
event.getText().add(item.mDescription);
}
```

`@Override`

```
protected void onPopulateNodeForVirtualView(
    int virtualViewId, AccessibilityNodeInfoCompat node) {
    final CustomItem item = getItem(virtualViewId);
    if (item == null) {
        throw new IllegalArgumentException("Invalid virtual view id");
    }
}
```

// 节点和事件文本和内容描述通常相同，因此我们将使用与之前完全相同的字符串。

```
node.setText(item.mDescription);
```

// 报告的边界应与用于在 `onDraw()` 中绘制项目的边界一致。它们还应该与 `getVirtualViewAt()` 和 `onTouchEvent()` 中执行的命中检测一致。

```
final Rect bounds = mTempRect;
int height = getHeight();
int width = getWidth();
if (item.mParent != null) {
    width = (int) (width * item.mParent.mBoundsInRoot.width());
    height = (int) (height * item.mParent.mBoundsInRoot.height());
}
scaleRectF(item.mBounds, bounds, width, height);
node.setBoundsInParent(bounds);
```

```
// Since the user can tap an item, add the CLICK action. We'll
// need to handle this later in onPerformActionForVirtualView.
node.addAction(AccessibilityNodeInfoCompat.ACTION_CLICK);
```

```
// This item has a checked state.
node.setCheckable(true);
node.setChecked(item.mChecked);
```

```
// Setup the hierarchy.
if (item.mParent != null) {
    node.setParent(CustomView.this, item.mParent.mId);
}
for (Integer id : item.mChildren) {
    node.addChild(CustomView.this, id);
}
```

```

    }
}

@Override
protected boolean onPerformActionForVirtualView(
    int virtualViewId, int action, Bundle arguments) {
    switch (action) {
        case AccessibilityNodeInfoCompat.ACTION_CLICK:
            // 点击处理应与 onTouchEvent() 一致。这确保了无论可访问性是
            // 打开还是关闭，视图的工作方式都是一样的。
            return onItemClicked(virtualViewId);
    }

    return false;
}

...

```

在自定义 [ExploreByTouchHelper](#) 中，重载必要的方法实现自定义 view 无障碍：

- **int getViewAt(float x, float y)**
- x, y 也就是我们处理 onTouchEvent 时获取的 x, y 当有触摸事件时，根据 x, y 返回当前是哪个结点，返回的 int 值由自己约定（和 getVisibleVirtualViews 方法对应，约定 index）
- **getVisibleVirtualViews(List<Integer> virtualViewIds)**
- 添加想要聚焦的 index 比如示例里 我只添加了有数据的分时柱子上，比如 8, 10, 22 就代表 8 点, 10 点, 22 点 这三个有数据的位置需要播报无障碍。
- **void onPopulateNodeForVirtualView(int virtualViewId, @NonNull AccessibilityNodeInfoCompat node)**
- 给虚拟 View 设置描述文本和边框。在 view 刷新时，会遍历 getVisibleVirtualViews 我们添加的结点 index，调用 onPopulateNodeForVirtualView 将焦点聚集，在 node 上设置要播报的内容，（还可以为其添加点击事件，添加的事件要在 onPerformActionForVirtualView 处理，而 setBoundsInParent 方法传入一个 rect, 边框是指无障碍模式下选中的区块边界。
- **onPerformActionForVirtualView**
- 提供交互，触发回调重绘控件

3.2.3.2、处理自定义触摸事件

自定义视图控件可能需要非标准的触摸事件行为。例如自定义控件可能使用 [onTouchEvent \(MotionEvent\)](#) 监听方法来检测 [ACTION_DOWN](#) 和 [ACTION_UP](#) 事件，并触发一个特殊的点击事件。为了保证无障碍服务的兼容性，处理自定义点击事件的代码必须如下：

1. 为解释点击操作生成适当的 [AccessibilityEvent](#)；
2. 启用无障碍服务来为不能使用触摸屏的用户执行自定义单击操作。

要有效的处理这些需求，代码应该重写 [performClick\(\)](#) 方法，需要调用该方法的父类实现，然后执行点击事件需要的任何操作。当检测到自定义点击操作时，代码应该调用 [performClick\(\)](#)。

onTouchEvent+performClick 代码示例

```
class CustomTouchView extends View {
    public CustomTouchView(Context context) {
        super(context);
    }

    boolean mDownTouch = false;

    @Override
    public boolean onTouchEvent(MotionEvent event) {
        super.onTouchEvent(event);
        //监听向下向上触摸事件
        switch (event.getAction()) {
            case MotionEvent.ACTION_DOWN:
                mDownTouch = true;
                return true;
            case MotionEvent.ACTION_UP:
                if (mDownTouch) {
                    mDownTouch = false;
                    performClick(); //调用该方法处理响应，启动无障碍服务为那些无法点
击的用户执行操作。
                }
                return true;
        }
        return false; //为其他触摸事件返回假。
    }

    @Override
    public boolean performClick() {
        //调用父类实现，父类会生成无障碍事件，并且在视图上调用 onClick() 监听器，如果
        任何;
        super.performClick();
        //在这里处理自定义点击操作
        return true;
    }
}
```

```

    }
}

//View.performClick
/**
 * Call this view's OnClickListener, if it is defined. Performs all normal
 * actions associated with clicking: reporting accessibility event, playing
 * a sound, etc.
 *
 * @return True there was an assigned OnClickListener that was called, false
 *         otherwise is returned.
 */
// NOTE: other methods on View should not call this method directly, but
performClickInternal()
// instead, to guarantee that the autofill manager is notified when necessary (as
subclasses
// could extend this method without calling super.performClick()).
public boolean performClick() {
    // We still need to call this method to handle the cases where performClick()
was called
    // externally, instead of through performClickInternal()
    notifyAutofillManagerOnClick();

    final boolean result;
    final ListenerInfo li = mListenerInfo;
    if (li != null && li.mOnClickListener != null) {
        playSoundEffect(SoundEffectConstants.CLICK);
        li.mOnClickListener.onClick(this);
        result = true;
    } else {
        result = false;
    }

    sendAccessibilityEvent(AccessibilityEvent.TYPE_VIEW_CLICKED);
    notifyEnterOrExitForAutoFillIfNeeded(true);
    return result;
}

```

4、无障碍服务适配 ([AccessibilityService](#))

Android 提供了标准的无障碍服务，包括 TalkBack，开发人员可以创建和发布自己的服务。构建和部署无障碍服务的能力在 Android 1.6（API 等级 4）被推出，并在 Android 4.0（API 等

级 14) 明显改善其功能。Android [支持库](#) 也被更新，Android 4.0 发布的支持库可以使得这些无障碍特性也能支持 android 1.6。

4.1、Manifests 声明和权限

提供无障碍服务的应用，必须在应用的声明（manifests）文件中明确声明，以便 android 系统把此应用程序作为无障碍服务处理。本节介绍无障碍服务的必需和可选设置。

4.1.1、无障碍服务声明

为了能被看作一个无障碍服务，必须在声明文件中的应用程序（application）元素中包含服务（service）元素（而不是活动（activity）元素）。此外，在服务（service）元素内，还必须包括无障碍服务的目标过滤器。为了兼容 Android 4.1 及更高版本的系统，声明文件中必须请求 [BIND_ACCESSIBILITY_SERVICE](#) 的权限

```
<manifest>
...
<uses-permission ... />
...
<application>
...
  <service android:name=".MyAccessibilityService"
    android:label="@string/accessibility_service_label"
    android:permission="android.permission.BIND_ACCESSIBILITY_SERVICE">
    <intent-filter>
      <action android:name="android.accessibilityservice.AccessibilityService" />
    </intent-filter>
  </service>
  <uses-permission android:name="android.permission.BIND_ACCESSIBILITY_SERVICE" />
</application>
</manifest>
```

4.1.2、无障碍服务配置

无障碍服务还必须提供一个配置，该配置指定了无障碍服务处理的无障碍事件类型和有关无障碍服务的附加信息。无障碍服务的配置被包含在 [AccessibilityServiceInfo](#) 类中。无障碍服务可以使用该类的实例创建一个配置，并在运行时使用 [setServiceInfo\(\)](#) 设置此配置。但是，使用该方法并不能配置所有的配置项。

从 Android 4.0 开始，可以在声明文件中包含一个<meta-data>元素，该元素指向一个配置文件。该配置文件允许开发者设置无障碍服务的所有内容

```
<service android:name=".MyAccessibilityService">
...
  <meta-data
```

```

        android:name="android.accessibilityservice"
        android:resource="@xml/accessibility_service_config" />
</service>

```

该 meta-data 元素指向的是开发者在应用资源目录下创建的 XML 文件
(`<project_dir>/res/xml/accessibility_service_config.xml`)。

```

<accessibility-service xmlns:android="http://schemas.android.com/apk/res/android"
    android:description="@string/accessibility_service_description"
    android:packageNames="com.example.android.apis"
    android:accessibilityEventTypes="typeAllMask"
    android:accessibilityFlags="flagDefault"
    android:accessibilityFeedbackType="feedbackSpoken"
    android:notificationTimeout="100"
    android:canRetrieveWindowContent="true"

    android:settingsActivity="com.example.android.accessibility.ServiceSettingsActivity"
    />

```

4.1.3、注册无障碍事件

无障碍服务配置参数最重要的功能之一是允许开发者指定无障碍服务能处理的无障碍事件类型。成功指定该信息使无障碍服务间能相互合作,并允许开发者灵活地处理特定应用的特定事件类型。事件过滤可以包含以下规则:

- **包名 (PackageNames):** - 指定想要无障碍服务处理的无障碍事件的应用程序包名。如果省略该参数,无障碍服务将被视为服务于任何应用程序的无障碍事件。在无障碍服务配置文件中,该参数可使用 `android:packageNames` 属性被设置为一个逗号分隔的列表,或使用 [AccessibilityServiceInfo.packageNames](#) 成员设置。
- **事件类型 (Event Types)** - 指定开发者想要无障碍服务处理的无障碍事件的类型。在无障碍服务配置文件中,该参数可使用 `android:accessibilityEventTypes` 属性设置为由 “ | ” 分隔的列表 (例如 `accessibilityEventTypes="typeViewClicked|typeViewFocused"`), 或者使用 [AccessibilityServiceInfo.eventTypes](#) 成员设置。

当创建无障碍服务时,开发者需要全面考虑无障碍服务能处理的无障碍事件,并只注册这些事件。因为用户可能同时激活多个无障碍服务,开发者创建的服务不能浪费在该服务不能处理的事件上。记住,其他的服务也可能会为了提供用户体验来处理这些事件。

4.2、无障碍服务

4.2.1、无障碍服务方法

一个无障碍服务必须继承 `AccessibilityService` 类，重写该类的以下方法。这些方法被 Android 系统调用时按顺序呈现，从服务被启动（`onServiceConnected()`），一直运行（`onAccessibilityEvent()`、`onInterrupt()`），到服务被关闭（`onUnbind()`）。

- `onServiceConnected()`
 - （可选）当系统成功连接到无障碍服务时，调用该方法。使用该方法为无障碍服务提供任何一次性的启动步骤，包含连接到用户反馈系统服务，例如音频管理或设备震动。如果想要在应用运行时设置无障碍服务的配置或做一次性的调整，可以在该方法中调用 `setServiceInfo()` 来实现。
- `onAccessibilityEvent()`
 - （必选）当系统检测到一个符合无障碍服务设定的无障碍事件过滤参数的事件时，该方法被系统回调。例如，当用户在应用程序中点击一个按钮或聚焦一个用户界面控件时，你的无障碍服务会为此应用程序提供反馈。此时，系统调用这个方法，传递相关联的无障碍事件，无障碍服务就可以翻译和使用该事件为用户提供反馈。在无障碍服务的生命周期中，该方法会被调用多次。
- `onInterrupt()`
 - （必选）当系统想要打断无障碍服务提供的反馈时该方法被调用，一般情况下是响应用户操作，如移动焦点到一个不同的控件。在无障碍服务的生命周期中，该方法会被调用很多次。
- `onUnbind()`
 - （可选）当系统想要关闭无障碍服务的时候，调用该方法。使用该方法去完成一次性的关闭进程，包括解除分配用户反馈系统服务，例如音频管理或者设备震动。

这些回调方法为无障碍服务提供基本架构。在 `AccessibilityEvent` 对象中怎样处理由 Android 系统提供的数据并为用户提供反馈，是开发者自己决定的。获取无障碍事件信息的更多内容，详见实现无障碍（[Implementing Accessibility](#)）培训。

4.2.2、获得事件详情

Android 系统通过 `AccessibilityEvent` 对象为无障碍服务提供用户界面交互的信息。Android 4.0 较大的扩展了无障碍服务能获取到的关于用户界面交互的信息量，这些信息是通过基于视图层次组合的无障碍事件提供。视图层次是用户界面元素的组合，该组合包含组件（父亲）和可能包含在其中的子元素（孩子）。因此，Android 系统可以提供无障碍事件的更丰富的详细信息，允许无障碍服务为用户提供更多有用的反馈。

系统将 `AccessibilityEvent` 传递到无障碍服务的 `onAccessibilityEvent()` 回调方法，因此，无障碍服务获得用户交互事件的信息。`AccessibilityEvent` 会提供事件的详细信息，包含 `AccessibilityEvent` 的执行类型、描述文本和其他信息。从 Android 4.0 开始（Android 4.0 以前的支持情况详见支持库（Support Library）中的 `AccessibilityEventCompat`），可以通过调用以下方法获得更多信息。

- `AccessibilityEvent.getRecordCount()` 和 `getRecord(int)`
 - 这些方法允许开发者获得一组 `AccessibilityRecord` 对象，该对象作为 `AccessibilityEvent` 的一部分，通过系统传递给开发者。该详情的级别为无障碍服务触发的事件提供了更多上下文信息。

- [`AccessibilityEvent.getSource\(\)`](#)
 - 该方法返回 [`AccessibilityNodeInfo`](#) 对象。该对象允许开发者请求查看触发无障碍事件的组件视图布局层次（父亲和孩子）。该特性允许无障碍服务访问事件的所有上下文内容，包含任何闭合视图或子视图的内容和状态。

重要: 从一个 [`AccessibilityEvent`](#) 访问视图层次的能力会将用户私人信息潜在的暴露给无障碍服务。基于该原因，无障碍服务必须在[无障碍服务配置 XML](#) 文件请求访问权限，该文件中需要包含 `canRetrieveWindowContent` 属性并将其设置为 `true`。如果 XML 文件不包含这些设置，调用 [`getSource\(\)`](#) 将会失败。

4.2.3、为用户采取措施

从 Android4.0（API 级别 14）开始，无障碍服务可以代表用户操作，包含改变输入焦点和选择（激活）用户界面元素。在 Android4.1（API 级别 16），操作的范围被扩展至包含滚动列表和与文本域交互。无障碍服务也可采取全局操作，如导航到主界面、按返回按钮、打开屏幕通知和最近应用列表。Android4.1 也包含新焦点类型，*无障碍焦点*，该焦点类型可让所有视觉元素能够被无障碍服务所选择。

4.2.3.1、手势监听

无障碍服务可以监听特定手势，并代表用户响应。该特性被添加在 Android4.1（API 级别 16）版本中，要求无障碍服务激活触摸浏览特性。无障碍服务可以通过设置 [`AccessibilityServiceInfo`](#) 实例的 `flags` 成员为 `FLAG_REQUEST_TOUCH_EXPLORATION_MODE`，请求该特性的激活

```
public class MyAccessibilityService extends AccessibilityService {  
    @Override  
    public void onCreate() {  
        getServiceInfo().flags =  
AccessibilityServiceInfo.FLAG_REQUEST_TOUCH_EXPLORATION_MODE;  
    }  
    ...  
}
```

一旦无障碍服务请求激活触摸浏览，如果该特性未打开，用户必须允许打开该特性。当该特性激活，无障碍服务将会通过 [`onGesture\(\)`](#) 回调方法接收无障碍手势的通知，并且为用户响应操作。

4.2.3.2、使用无障碍操作

无障碍服务可以代替用户操作，让用户与应用的交互更简单有效。无障碍服务的这个能力在 Android4.0（API 级别 14）被加入，在 Android4.1（API 级别 16）被显著扩展。

为了代替用户操作，无障碍服务必须[注册](#)来接收来自某些或者更多应用中的事件，并在[无障碍服务配置文件](#)中设置 [`android:canRetrieveWindowContent`](#) 属性为 `true`，来请求查看应用程序

的内容。当接收到事件，无障碍服务可以使用 [getSource\(\)](#) 方法在事件中检索 [AccessibilityNodeInfo](#) 对象。使用 [AccessibilityNodeInfo](#) 对象你的无障碍服务可以浏览视图层次来判定采取什么操作并使用 [performAction\(\)](#) 为用户操作。

```
public class MyAccessibilityService extends AccessibilityService {

    @Override
    public void onAccessibilityEvent(AccessibilityEvent event) {
        // 获得事件的资源节点
        AccessibilityNodeInfo nodeInfo = event.getSource();

        //使用事件和节点信息判定为用户执行什么样的操作。
        nodeInfo.performAction(AccessibilityNodeInfo.ACTION_SCROLL_FORWARD);

        //循环 nodeInfo 对象
        nodeInfo.recycle();
    }
    ...
}
```

[performAction\(\)](#) 方法允许无障碍服务在应用中执行操作。如果无障碍服务需要去执行全局操作，例如导航到主屏幕、按返回键、打开屏幕通知或者最近应用列表等，使用 [performGlobalAction\(\)](#) 方法。

4.2.3.3、使用焦点模式

Android 4.1 (API 级别 16) 引入了一种新的用户界面焦点，被称为 *无障碍焦点 (Accessibility Focus)*。这种类型的焦点可用于在无障碍服务中，选择任何视觉用户界面元素并操作该元素。该焦点类型不同于 *输入焦点 (Input Focus)*。当用户输入字符，点击键盘上的 **enter** 键或 D-pad 控制器的中心按钮时，输入焦点决定屏幕上哪个元素接收输入。

无障碍焦点与输入焦点是完全分开和独立的。事实上，用户界面中的一个元素有输入焦点，同时另一个元素有无障碍焦点，这种情况是可能出现的。无障碍焦点的目的是，为无障碍服务提供一种与屏幕上视觉元素进行交互的方法，而不管该元素是否是系统层面的可输入聚焦元素。开发者可以在无障碍手势测试中看到无障碍焦点。更多关于该特性的信息，详见[测试手势导航](#)。

注：当元素具有输入焦点，使用无障碍焦点的无障碍服务有责任整合当前输入焦点。如果服务没有整合输入焦点和无障碍焦点，有可能会在应用中引起问题，问题是当采用特定操作时，需要将输入焦点固定在特定的位置。

无障碍服务可以使用 [AccessibilityNodeInfo.findFocus\(\)](#) 方法来判定哪些用户界面元素有输入焦点或无障碍焦点，也可以使用 [focusSearch\(\)](#) 方法来检索哪些元素可以用输入焦点选择。最后，可以使用 [performAction\(AccessibilityNodeInfo.ACTION_SET_ACCESSIBILITY_FOCUS\)](#) 方法来设置无障碍焦点。

4.3、注意事项

4.3.1、Android12 无障碍服务 isAccessibilityTool 管控

1) 背景

Android 12 中, Google 增加了对三方应用无障碍服务的严格管控, 因为很多三方应用无障碍服务并不是真正为无障碍人群服务的, 只是利用无障碍服务在非法获取用户信息。为了管控这些行为, Google 在新一台平台中新增了 `Flag AccessibilityServiceInfo#isAccessibilityTool`, 用于表示当前无障碍服务是否是一个无障碍工具; 如果没有配置 `isAccessibilityTool` 配置, 该无障碍服务启动的时会显示前台通知, 告知用于当前无障碍服务可能会访问用户数据, 并提供选项卸载该无障碍服务。

对于声明了该 `Flag` 的应用, 被系统信任还需要满足以下条件之一:

- a 系统应用 (`ApplicationInfo.FLAG_SYSTEM`)
- b 非系统应用, 则必须来源于受信任的安装源

受信任的安装源必须是系统 app, 并且也要满足以下两个条件之一:

- a 如果系统无障碍信任安装源的名单为空, 安装源的应用不是当前系统的默认安装器
- b 如果系统无障碍信任安装源的名单不为空, 安装源的应用必须要在该名单中

2) 兼容性影响

如果应用处于以下条件:

- 1 没有无障碍服务配置中声明 `android:isAccessibilityTool="true"`
- 2 声明了 `android:isAccessibilityTool="true"` 同时来源于不受信任的安装源

那么应用在 Android 12 的版本上运行时, 会显示提示信息提示用户该无障碍服务正在运行, 并提供选项卸载该应用。

5、常见问题

5.1. 如何判断当前无障碍 talkback 是否开启?

通过获取当前系统开启的所有辅助功能服务的 `Settings` 值, 判断其中有没有包含 `talkback`, 从而判断 `talkback` 开关有没有开启, 参考 [3.2.1、开启状态判断](#)

5.2. 播报如何加标题? 如: "xxx, 标题"

使用 `android:accessibilityHeading` 控制, 参考: [3.1.1.5、标题](#)

5.3. 如何去掉播报 “点按两次即可激活” ？

在 `onInitializeAccessibilityNodeInfo` 方法内部通过 `AccessibilityNodeInfo.removeAction` 移除对应无障碍操作（`AccessibilityNodeInfo.AccessibilityAction.ACTION_CLICK`）即可去掉对应操作提示，参考 [3.2.2.4、填充无障碍事件](#)

去掉 “长按” 播报：

`info.removeAction(AccessibilityNodeInfo.AccessibilityAction.ACTION_LONG_CLICK);`

```
ViewCompat.setAccessibilityDelegate(holder.itemView, onInitializeAccessibilityNodeInfo(host, info) → {
    super.onInitializeAccessibilityNodeInfo(host, info);
    info.setClickable(false);
    info.removeAction(AccessibilityNodeInfoCompat.AccessibilityActionCompat.ACTION_CLICK);
});
```

5.4. 如何直接播报文本？

使用 `mView.announceForAccessibility(java.lang.CharSequence)` 直接触发朗读即可，参考 [3.1.4、朗读控制](#)

5.5. 如何修改 “点按两次即可 xx” 的 ACTION 描述？

参考文档中 demo 有实现样例，核心为通过 [ViewCompat.replaceAccessibilityAction\(\)](#) 来重新定义相应的无障碍操作。

```
AccessibilityNodeInfoCompat.AccessibilityActionCompat actionCompat = new
AccessibilityNodeInfoCompat.AccessibilityActionCompat(AccessibilityNodeInfoCompat.A
CTION_LONG_CLICK, "单指双击两次 haha ");
```

```
ViewCompat.replaceAccessibilityAction(mButton, actionCompat,
    "弹出弹出式窗口", new AccessibilityViewCommand() {
        @Override
        public boolean perform(@NonNull View view, @Nullable
CommandArguments arguments) {
            Toast.makeText(BasicElementAccessibilityActivity.this, "长按
执行了", Toast.LENGTH_SHORT).show();
            mButton.callOnClick();
            return true;
        }
    });
ViewCompat.replaceAccessibilityAction(mButton,
```

```
AccessibilityNodeInfoCompat.AccessibilityActionCompat.ACTION_SCROLL_TO_POSITION,
    "弹出弹出式窗口", new AccessibilityViewCommand() {
        @Override
```

```

        public boolean perform(@NonNull View view, @Nullable
CommandArguments arguments) {
            Toast.makeText(BasicElementAccessibilityActivity.this, "滑动
执行了", Toast.LENGTH_SHORT).show();
            mButton.callOnClick();
            return true;
        }
    });

```

5.6. 如何转换无障碍控件类型或修改控件类型描述

转换无障碍控件类型可以使用 `AccessibilityNodeInfoCompat.setClassName` 方法进行转换。

修改控件类型描述可以使用 `AccessibilityNodeInfoCompat.setRoleDescription` 方法，或者 `AccessibilityNodeInfo.getExtras().putCharSequence("AccessibilityNodeInfo.roleDescription", "我是控件类型")`；参考 [3.2.2.4、填充无障碍事件](#)

5.7. 如何去掉冗余的焦点？

xml 里这样写：`android:importantForAccessibility="no"`

java 里这样写：

```
mView.setImportantForAccessibility(View.IMPORTANT_FOR_ACCESSIBILITY_NO);
```

5.8. 如何清除已选中的无障碍焦点？

```
mView.performAccessibilityAction(AccessibilityNodeInfo.ACTION_CLEAR_ACCESSIBILITY_FOCUS, null);
```

5.9. 如何将焦点转移到父布局？

父布局加：`android:focusable="true"` `android:focusableInTouchMode="true"`

如若子 view 不需要焦点则加：`android:importantForAccessibility="no"`

5.10. 如何主动获取焦点？

可以通过在 UI 线程延迟发送事件的方式来实现某个控件主动获取焦点，参考：[3.1.3.2、主动获取焦点](#)

5.11. 如何控制 talkback 焦点顺序

talkback 输入无障碍服务,在 talkback 模式下使用 `android:accessibilityTraversalAfter` 和 `android:AccessibilityTraversalBefore` 这两个属性来控制焦点导航。参考 [3.1.3.3、焦点导航](#)。

5.12. web 相关 H5 页面如何适配?

H5 页面适配主要参考以下几个资料:

- [信息无障碍协会 Web 技术文档](#)
- [Web 如何适配无障碍?](#)
- [小程序如何适配无障碍?](#)

5.13. 如何打印节点树?

1. 打印节点树

1.1 启用节点树调试功能

找到启用节点树调试功能项目,开启即可

路径: TalkBack 设置->高级设置->开发者设置

▼ 1.2 设置日志输出级别为

将日志输出级别设置为VERBOSE

路径: TalkBack 设置->高级设置->开发者设置

1.3 设置打印节点数手势

设置打印节点的自定义手势,这里将四指点按作为手势

路径: TalkBack 设置->自定义手势

1.4 在Logcat中查看日志信息

由于TalkBack的日志信息非常多,可筛选“talkback”和“TreeDebug”为关键字

6、参考文档

文档	来源	备注
----	----	----

《 TalkBack 设计规范 》	vivo	vivo 策划部门拟制
《 Android 无障碍功能概览 》	Google	Android 官方文档
《 改进应用无障碍功能要遵循的原则 》	Google	Android 官方文档
《 Android 无障碍技术文档 》	Blog	信息无障碍协会
《 让应用无障碍 》	Blog	翻译 Google 文档
《 无障碍学习整理（基于 talkback） 》	Blog	
《 Android 无障碍适配指南 》	Blog	